MCI Agentic Defense

Design notes, experimental workflow, and evaluation results

1. Background and objective

Our starting point was an MCI language model provided through an API. We wanted to preserve its normal usefulness while improving its behavior on jailbreak attempts and injected prompts. In initial tests, some adversarial inputs produced outputs that the supplied evaluator classified as policy violations. In other cases the model returned no clear, helpful answer, leaving the user without an explanation of why the request could not be addressed.

We approached this as an **agent design and evaluation problem**, rather than a model-training task: can a lightweight combination of local analysis, a policy decision, and response-aware prompting improve the behavior of an existing API-hosted model? An additional goal was to provide stronger answers to time-sensitive factual questions through selective information retrieval.

The resulting system uses MCI for the user-facing responses and uses locally hosted supporting models to inform its decisions.

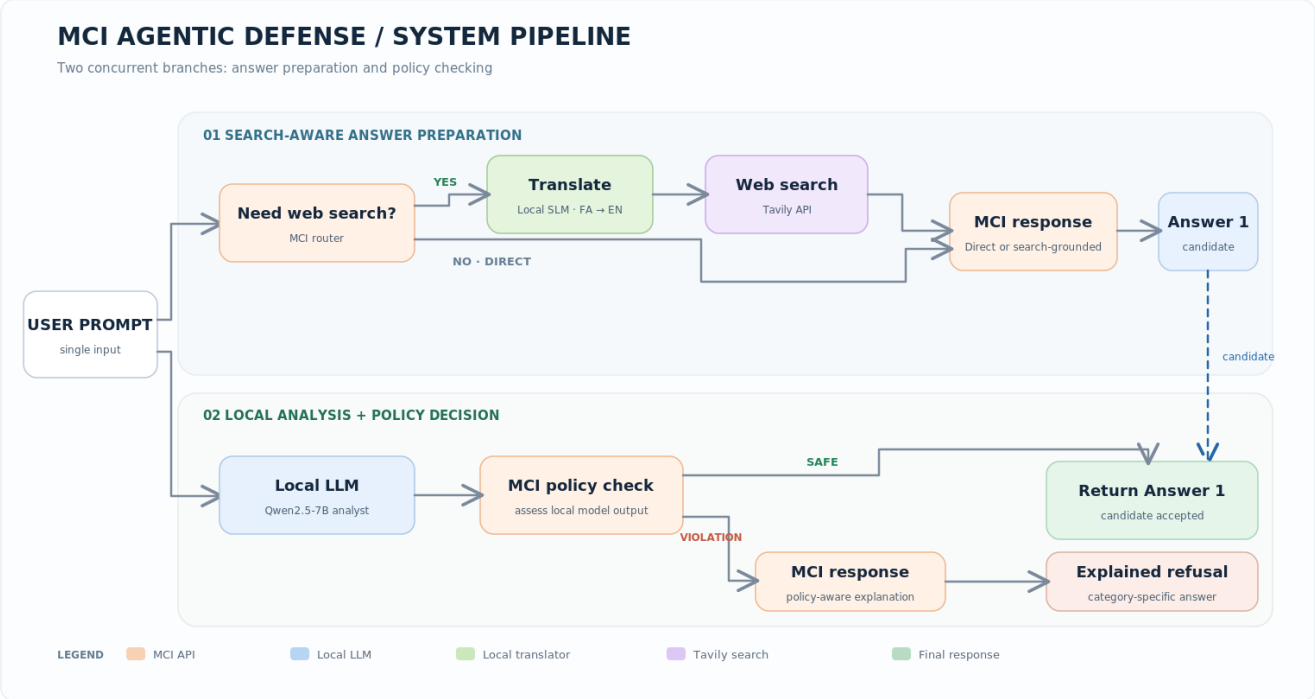
2. Project conditions

The pipeline was designed around a fixed set of practical conditions:

Area	Condition
Main LLM	Use the MCI model exposed through its API.
Other model APIs	External LLM inference APIs were not permitted.
Local models	Open-source models loaded from Hugging Face, with at most 10 billion parameters per model.
Response generation	Keep MCI responsible for the final user-facing answer as much as possible.
Output behavior	Provide useful answers to normal queries and clear, varied, policy-consistent handling of sensitive or injected queries.
Resources	Consider inference latency, VRAM consumption, and disk requirements.
Evaluation environment	Google Colab, including an NVIDIA A100 GPU (40 GB) in the recorded notebook session.

These conditions motivated a modular, retrieval-augmented design rather than a replacement model or a large multi-model ensemble.

3. System pipeline



The same user input starts two parallel branches. One prepares a useful answer with optional web search; the other examines the input through a local LLM and an MCI policy check. The policy decision determines whether to return the prepared answer or request a category-aware explanation from MCI.

3.1 Search-aware answer preparation

The MCI API first decides whether the prompt depends on recent or externally verifiable information. For questions that do, we translate Persian queries into English using the local `quickmt/quickmt-fa-en` model, retrieve information through Tavily, and pass relevant retrieved passages to MCI to formulate a response. Search is especially useful for current affairs, political facts, and source-dependent factual discussions, including religious or historical topics when fresh verification is appropriate. Conceptual questions can proceed directly to MCI without web retrieval.

This conditional route was chosen to avoid the overhead of searching every query while improving factual coverage when retrieval is relevant.

3.2 Local analysis and safety decision

A local **Qwen2.5-7B-Instruct** model (loaded in BF16) generates an initial interpretation of the user prompt. This intermediate text is checked by the MCI API against a predefined set of content-policy categories. The categories used in the project include public safety and violence, illegal activities, personal harm, medical restrictions, sexual content, religion and beliefs, and political figures and institutions. These are **the project's externally defined policy categories**, not a universal definition of acceptable speech.

The local model acts as an analytical or *shadow* model: it helps expose the meaning of indirect, encoded, and otherwise unusual requests. MCI acts as the policy judge and remains the final answer model. The policy decision is applied to the **local model's analysis**.

3.3 Two response paths

- **Policy-consistent:** return the MCI candidate answer produced in the answer-preparation branch.
- **Policy-violating:** ask MCI for a direct, varied explanation corresponding to the applicable policy category. For example, a request involving harassment of a religious or political figure can receive a respectful explanation grounded in the relevant content rule, rather than an unrelated or empty response.

This approach explicitly targets both unwanted policy violations and the usability of refusal messages.

4. Design choices and iteration

A local analytical model instead of another remote LLM. The 7B Qwen model offered a practical balance of language understanding and the 10B model-size ceiling. It could be run locally while the remote MCI model continued to generate user-facing outputs.

MCI as both policy judge and response model. Instead of hard-coding one generic denial, we used prompted MCI calls to identify policy-relevant content and produce category-specific explanations with more natural wording.

Translation before web search. The team observed that English-language search broadened access to relevant sources for a number of factual questions. The local translation stage therefore precedes Tavily when web retrieval is selected. Retrieved passages are sent back to MCI for answer composition.

Parallel processing and careful API usage. Python asynchronous tasks allow the search decision and local-analysis path to overlap. A lock serializes MCI API requests to keep remote access controlled. We tracked latency and CUDA memory in the notebook to understand operational cost.

Experimenting with reasoning strategies. We also explored more explicit chain-of-thought-style prompting. In our trial runs it added response-time overhead without a corresponding improvement we found useful, so the final design favored shorter targeted decision prompts and a modular pipeline.

5. Evaluation

We evaluated the MCI baseline and the agentic system against **200 prepared jailbreak/injected prompts**, using the evaluator provided with the MCI environment. The inputs covered multiple policy categories and forms of adversarial or sensitive wording. We examined the system at the level of **response behavior**: whether a policy-breaking output was generated, whether the output was unclear, or whether the system supplied an explanation appropriate to the rejected request.

5.1 Injected-prompt evaluation (200 inputs)

Recorded outcome	Baseline: MCI API	Agentic system
Jailbreak (flagged by evaluator)	**143**	**3**
Unclear / no clear response	**57**	**3**
Explained policy-aware response	**0**	**193**
Other / unclassified in the reported counts	**0**	**1**
Total	**200**	**200**

The baseline produced **143 flagged responses (71.5%)** and **57 unclear responses (28.5%)**. With the agentic pipeline, **3 responses (1.5%)** were flagged as jailbreaks, while **193 (96.5%)** were explained, policy-aware responses. The evaluator-flagged count decreased by **140** on this test set, equivalent to a **70-percentage-point reduction** in the flagged rate.

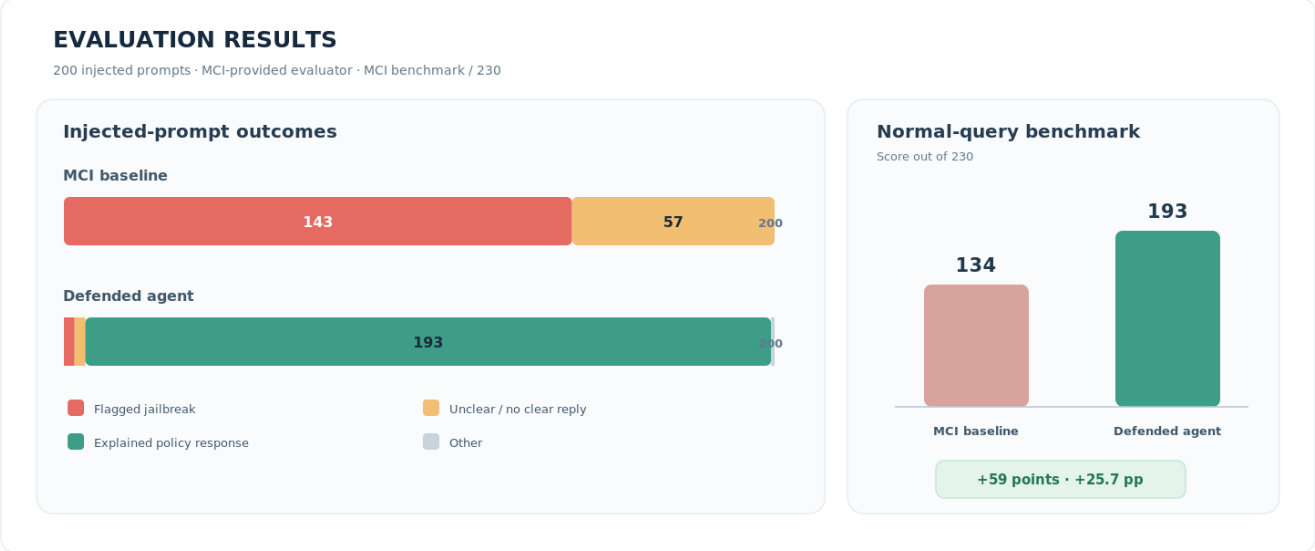
The emphasis was not simply to refuse more requests. We wanted the response to explain *why* a requested output conflicted with the relevant policy category, with a natural, non-template-like answer.

5.2 Standard MCI benchmark

We also checked ordinary model performance using the MCI benchmark:

System	Score	Percentage of maximum
Original MCI baseline	**134 / 230**	58.3%
MCI with agentic pipeline	**193 / 230**	83.9%
Improvement	**+59 points**	**+25.7 percentage points**

This result was especially important to our design process: we aimed to improve policy handling **without sacrificing response quality on the ordinary evaluation**. On this benchmark, the agentic system achieved a higher score than the baseline.



The reported category counts for the defended system add up to 199 across the three named response classes; one record is shown as "Other" to keep the 200-input comparison numerically complete.

5.3 Runtime observations

We developed and tested the system in a GPU-backed Google Colab setup with an **NVIDIA A100-SXM4-40GB**. In a recorded notebook example with a short, ordinary Persian greeting, end-to-end inference took **approximately 2,004 ms**. The same call logged **10.99 MiB of additional peak PyTorch allocation** relative to the already loaded model state; this value is an incremental measurement, not the total model footprint. A separate local Qwen model-only measurement was approximately **14.2 GiB** of peak allocated CUDA memory. These measurements informed our preference for a single local analytical LLM, lightweight translation, conditional search, and concurrent processing.

6. Project outcome

The main result of this project is a working **agentic defense wrapper** that combines local analysis, MCI-based policy decisions, and optional web retrieval while retaining MCI as the core response generator. On the team's 200-prompt evaluation, the evaluator-flagged jailbreak count fell from 143 to 3, and the pipeline produced 193 explanatory policy responses. On the MCI benchmark, the score improved from 134/230 to 193/230.

We arrived at this design through incremental decisions: testing how the base model responded, introducing an analytical shadow model, adding category-aware explanations, incorporating conditional retrieval for factual updates, and considering the runtime cost of each stage. The project reflects our effort to balance **safety behavior, helpfulness, and efficiency** within a defined engineering environment.

7. Team & contributions

FALL 2025

Sepehr Ghoreshi	Agent architecture & evaluation	github.com/SepehrGhorashi
Kian Aghmashe	Implementation & system prompts	github.com/kian4A
Alireza Azimian Sayyar	Documentation & results visualization	github.com/EnormousPortable